SVP Program Report

Calvin Dantis zmzf47

1 Reasoning

Despite enumeration's inefficiency, it outperforms lattice reduction sacrificing scalability. This approach is not applicable if the basis contains linearly dependent vectors; in these cases lattice reduction is used in tandem to enumeration.

FP-LLL[1] was chosen for its simplicity, speed and accuracy for low-dimensional lattices. This was chosen over LLL as the input basis could consist of real values and double-precision is employed, increasing accuracy.

SE enumeration[1] is an exact SVP algorithm however it is inefficient with low orthogonality bases and incorrect if there exists linearly dependent vectors; this is eliminated by using it with lattice reduction.It also utilises less space compared to sieving despite being asymptotically slower.

2 Matrix

The matrix class expresses an n x m matrix where a vector contains m double arrays of size n, when a matrix is initialised, m=n. Arrays are more efficient in this case as there is no dynamic memory allocation overhead. A vector is used to contain the arrays as memory may need to be dynamically allocated if a linearly dependent column is found. Its destructor also safely destroys the matrix to prevent memory leaks.

3 Parser

The column dimension is calculated by square rooting the number of arguments as each basis value corresponds to a unique argument in a square input basis; this then initialises a square matrix which is sequentially filled by parsing each argument with modular arithmetic. Checks are implemented to catch input errors and prevent solver issues.

4 SVP Solver

Containers for matrix information, including Gram-Schmidt norms and coefficients for each column, are defined. To minimise space, the coefficient vector only stores $(m(m+1))/2$ elements and an index function is used to locate a column's coefficients. The norms are pre-processed before reduction and updated only when a column changes to minimise expensive dot product calculations.

The enumeration algorithm closely follows Masaya Yasuda's optimised version[2], minimising arrays and eliminating redundancies. As it is executable without lattice reduction, Gram-Schmidt (GS) information has to be initialised separately, if a GS norm of 0 is found, indicating linear dependence, it aborts the initialisation so the basis can be reduced instead.

The reduction algorithm closely mirrors the original FP-LLL with accommodations for linear dependence and floating-point error countermeasures using rounding. Data referencing optimisations greatly improve performance: columns and coefficient indexes are referenced and stored before they are iterated on for quicker access. Zero vectors are checked using the pre-processed norms instead and the iteration counter isn't reset to reduce redundancy and the GS information calculated during reduction is also reused for enumeration to minimise redundancy.

5 Complexity

Scalability is the largest drawback, the time complexity of parsing is $\Theta(n^2)$ due to the constant n^2 arguments. Enumeration's super-exponential[2] running time has minimal impact at low dimension and reduction runs in $O(n^6\log^3(B))$[3] where B is the largest norm of input basis but the following enumeration will likely run in linear time as lattice reduction typically contains the shortest vector. Resulting in super-exponential time complexity unless a linearly dependent column is present, in which case the average running time becomes $O(n^6\log_3(B))$.

The program's space complexity is $\Theta(n^2)$ due to matrix storage, unavoidable without sacrificing correctness; however, the solver's space complexity is $O(n)$, as lattice reduction may reduce space required for enumeration if a linearly dependent column is detected.

6 Performance

Several measurements of the performance of the program have been recorded at different dimensions with varying amounts of linear dependent columns recording the execution time and storage utilisation.

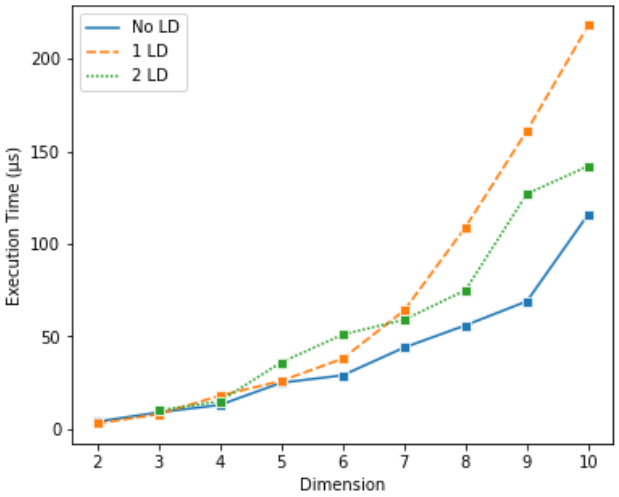

Figure 1 Execution times at different dimensions

The average execution time of 1,000,000 executions was recorded where each matrix is randomly generated with 2 decimal point values between (-1000, 1000). The process of writing to the file is not measured. The performance is highest when there are no linearly dependent columns as the slower lattice reduction isn't executed however if there does exist a linearly dependent column, then more linearly dependent columns decrease execution time as more columns are eliminated for enumeration, saving time and space.

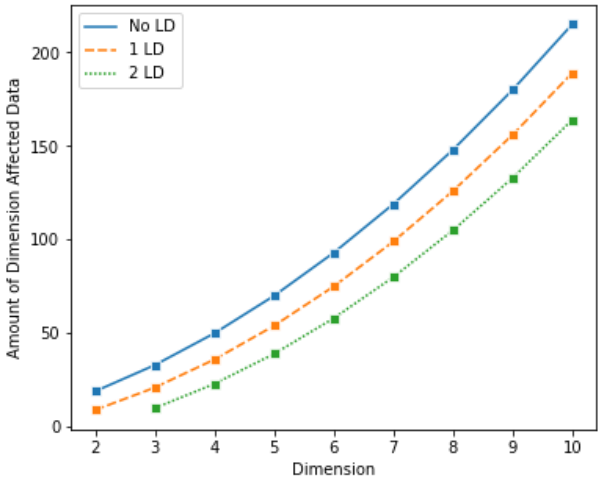

Figure 2 Amount of data stored in containers affected by input dimension

Various containers are affected by input dimension, these can be modelled as an equation to determine a general trend for storage utilisation. The parser utilises n^2 space, the solver requires $2m+(m(m+1))/2$ space and enumeration utilises an additional $5m$ space. m can be expressed as $(n-n_{ld})$ where n_{ld} is the number of linearly dependent columns as they are removed during reduction.

$$(n - n_{ld})(n + \frac{(n-n_{ld}-1)}{2} + 7)$$

Linear independence, while slowing execution, frees memory during reduction, displaying a speed-memory tradeoff.

7 References

[1] C. P. Schnorr and M. Euchner, "Lattice basis reduction: Improved practical algorithms and solving subset sum problems," *Mathematical Programming*, vol. 66, no. 1–3, pp. 181–199, 1994. doi:10.1007/bf01581144

[2] M. Yasuda, "A survey of solving SVP algorithms and recent strategies for solving the SVP Challenge," *International Symposium on Mathematics, Quantum Theory, and Cryptography*, pp. 189–207, 2020. doi:10.1007/978-981-15-5191-8_15

[3] S. D. Galbraith, "Lattice Basis Reduction," in *Mathematics of public key cryptography*, Cambridge: Cambridge University Press, 2012, pp. 365–381